

5 Client-side Programming

5.1 Introduction to Scripting

JavaScript: Scripting language used to enhance the functionality and appearance of web pages

Hello World

```
<html>
  <head>
    <title>A first Program in JavaScript</title>
    <script type = "text/javascript">
      document.writeln("<h1>Welcome to JavaScript Programming!</h1>");
    </script>
  </head><body></body>
</html>
```

Welcome to JavaScript Programming!

script element

- indicates that what follows is part of a script
- Browser's **document** object represents the HTML5 document currently being displayed in the browser
- Document object's `writeln` method writes a line of HTML5 text in the HTML5 document

```
<html>
  <head>
    <title>Printing a Line with Multiple Statements</title>
    <script type = "text/javascript">
      <!--
      document.write("<h1 style = 'color: magenta'>");
      document.write("Welcome to JavaScript " +
        "Programming!</h1>");
      // -->
    </script>
  </head><body></body>
</html>
```

Welcome to JavaScript Programming!

The window Object

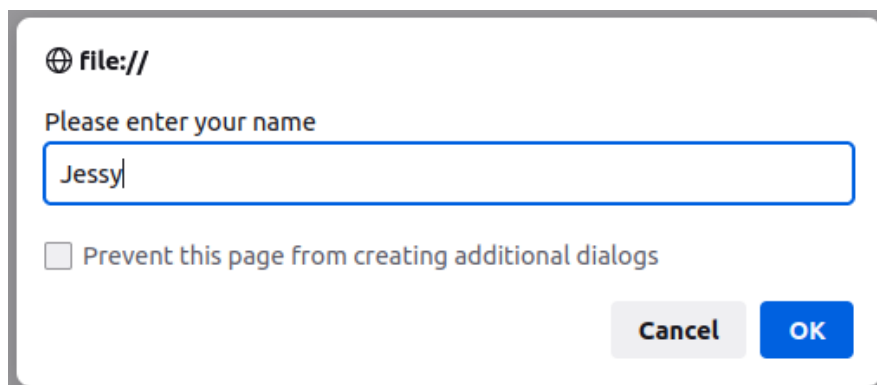
alert dialog

```
<html>
  <head>
    <title>Printing Multiple Lines in a Dialog Box</title>
    <script type = "text/javascript">
      <!--
        window.alert("Welcome to\nJavaScript\nProgramming!");
      // -->
    </script>
  </head><body></body>
</html>
```



prompt dialog

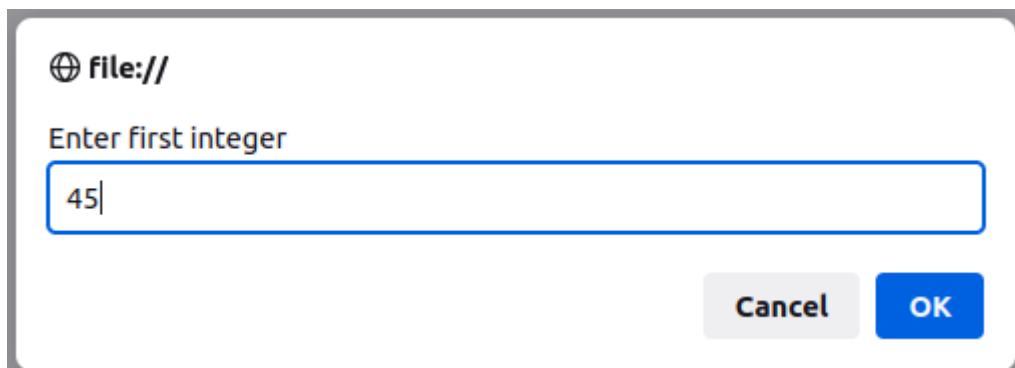
```
<html>
  <head>
    <title>Printing Multiple Lines in a Dialog Box</title>
    <script type = "text/javascript">
      <!--
        var name;
        name = window.prompt("Please enter your name");
        document.writeln("<h1>Hello " + name + ", welcome to JavaScript
programming!</h1>");
      // -->
    </script>
  </head><body></body>
</html>
```



Hello Jessy, welcome to JavaScript programming!

prompt dialog II

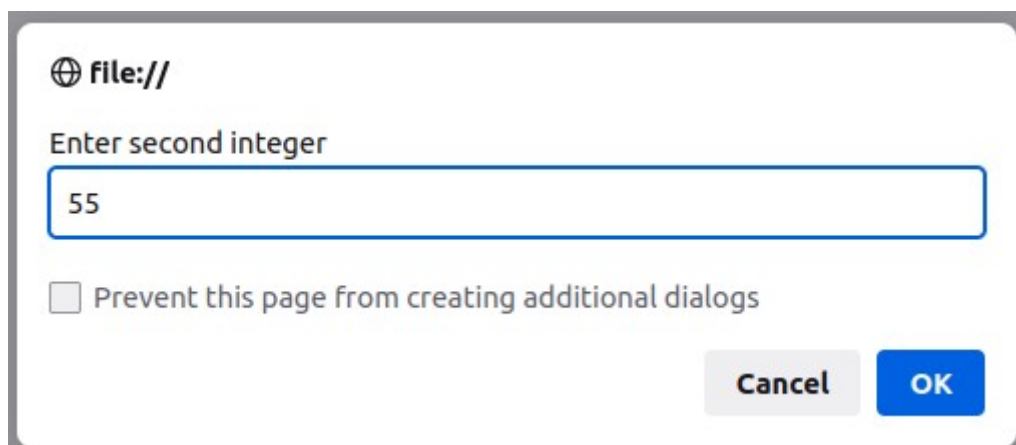
```
<html>
<head>
  <title>Printing Multiple Lines in a Dialog Box</title>
  <script type = "text/javascript">
    <!--
      var firstNumber;
      var secondNumber;
      var number1;
      var number2;
      var sum;
      firstNumber = window.prompt("Enter first integer");
      secondNumber = window.prompt("Enter second integer");
      // convert numbers from strings to integers
      number1 = parseInt(firstNumber);
      number2 = parseInt(secondNumber);
      sum = number1 + number2;
      document.writeln("<h1>The sum is " + sum + "</h1>");
    // -->
  </script>
</head><body></body>
</html>
```



file:/> Enter first integer

45

Cancel OK



file:/> Enter second integer

55

☐ Prevent this page from creating additional dialogs

Cancel OK

The sum is 100

5.2 JavaScript Control Statements

Similar to other languages

While loop example

```
<html>
<head>
  <title>Printing Multiple Lines in a Dialog Box</title>
  <script type = "text/javascript">
    var total;
    var gradeCounter;
    var grade; // grade typed by user (as a string)
    var gradeValue; // grade value (converted to integer)
    var average;
    var gradeCount;
    total = 0;
    gradeCount = 5;
    gradeCounter = 1;
    while (gradeCounter <= gradeCount) {
      grade = window.prompt("Enter integer grade:", "0");
      gradeValue = parseInt(grade);
      total = total + gradeValue;
      gradeCounter = gradeCounter + 1;
    }
    average = total / gradeCount;
    document.writeln("<h1>Class average is " + average + "</h1>");
  </script>
</head><body></body>
</html>
```

Sentinel-controlled while loop example

```
<html>
<head>
  <title>Printing Multiple Lines in a Dialog Box</title>
  <script type = "text/javascript">
    var total;
    var gradeCounter;
    var grade; // grade typed by user (as a string)
    var gradeValue; // grade value (converted to integer)
    var average;
    var gradeCount;
    total = 0;
    gradeCount = 0;
    gradeCounter = 0;
    while (gradeValue != -1) {
      grade = window.prompt("Enter integer grade:", "0");
      gradeValue = parseInt(grade);
      total = total + gradeValue;
      gradeCounter = gradeCounter + 1;
    }
    average = total / gradeCounter;
    document.writeln("<h1>Class average is " + average + "</h1>");
  </script>
</head><body></body>
</html>
```

While, for, and do while loops with inline styling

```
<html>
  <head>
    <title>Printing Multiple Lines in a Dialog Box</title>
    <script type = "text/javascript">
      var counter = 1;
      while (counter <= 7) {
        document.writeln("<p style = 'font-size: " + counter + "ex'>HTML5 font
size " + counter + "ex</p>");
        ++counter; // increment
      }
    </script>
  </head><body></body>
</html>
```

```
<html>
  <head>
    <title>Printing Multiple Lines in a Dialog Box</title>
    <script type = "text/javascript">
      for (var counter = 1; counter <= 7; ++counter)
        document.writeln("<p style = 'font-size: " + counter + "ex'>HTML5 font
size " + counter + "ex</p>");
    </script>
  </head><body></body>
</html>
```

```
<html>
  <head>
    <title>Printing Multiple Lines in a Dialog Box</title>
    <script type = "text/javascript">
      var counter = 1;
      do {
        document.writeln("<p style = 'font-size: " + counter + "ex'>HTML5 font
size " + counter + "ex</p>");
        ++counter;
      } while (counter <= 7)
    </script>
  </head><body></body>
</html>
```

HTML5 font size 1ex

HTML5 font size 2ex

HTML5 font size 3ex

HTML5 font size 4ex

HTML5 font size 5ex

HTML5 font size 6ex

HTML5 font size 7ex

```

<html>
  <head>
    <meta charset = "utf-8">
    <title>Calculating Compound Interest</title>
    <style type = "text/css">
      table      { width: 300px;
                    border-collapse: collapse;
                    background-color: lightblue; }
      table, td, th { border: 1px solid black;
                      padding: 4px; }
      th         { text-align: left;
                    color: white;
                    background-color: darkblue; }
      tr.oddrow   { background-color: white; }
    </style>
    <script>

      var amount; // current amount of money
      var principal = 1000.00; // principal amount
      var rate = 0.05; // interest rate

      document.writeln( "<table>" ); // begin the table
      document.writeln(
        "<caption>Calculating Compound Interest</caption>" );
      document.writeln(
        "<thead><tr><th>Year</th>" ); // year column heading
      document.writeln(
        "<th>Amount on deposit</th>" ); // amount column heading
      document.writeln( "</tr></thead><tbody>" );

      // output a table row for each year
      for ( var year = 1; year <= 10; ++year )
      {
        amount = principal * Math.pow( 1.0 + rate, year );

        if ( year % 2 !== 0 )
          document.writeln( "<tr class='oddrow'><td>" + year +
                             "</td><td>" + amount.toFixed(2) + "</td></tr>" );
        else
          document.writeln( "<tr><td>" + year +
                             "</td><td>" + amount.toFixed(2) + "</td></tr>" );
      } //end for

      document.writeln( "</tbody></table>" );

    </script>
  </head><body></body>
</html>

```

Calculating Compound Interest

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

5.3 JavaScript Functions

Example 1: Max of three numbers; makes use of inbuilt method `Math.max()` and programmer-defined function `maximum()`

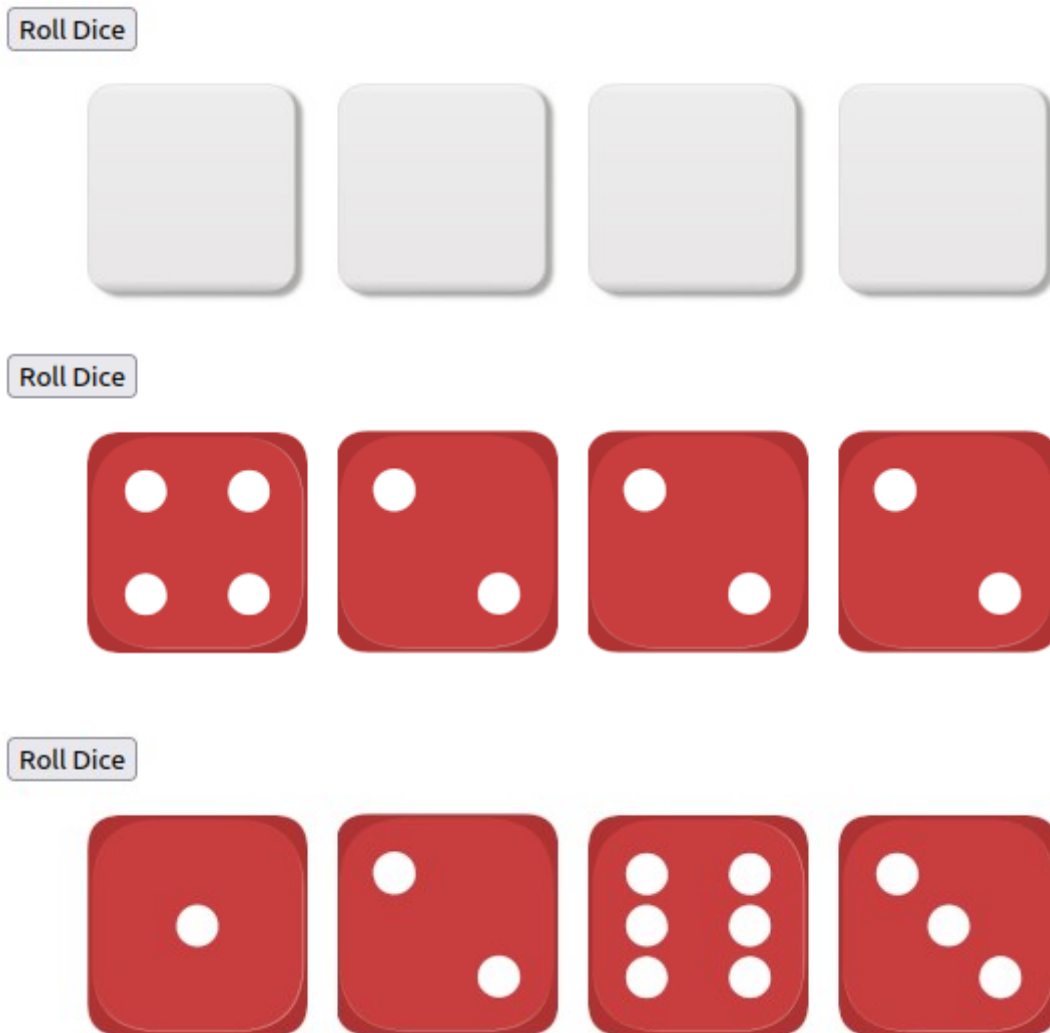
```
<html>
<head>
  <title>Maximum of Three Values</title>
  <style type = "text/css">
    p { margin: 0; }
  </style>
  <script>
    var input1 = window.prompt("Enter first number", "0");
    var input2 = window.prompt("Enter second number", "0");
    var input3 = window.prompt("Enter third number", "0");
    var value1 = parseFloat(input1);
    var value2 = parseFloat(input2);
    var value3 = parseFloat(input3);
    var maxValue = maximum(value1, value2, value3);
    document.writeln("<p>First number: " + value1 + "</p>" +
      "<p>Second number: " + value2 + "</p>" +
      "<p>Third number: " + value3 + "</p>" +
      "<p>Maximum is: " + maxValue + "</p>");
    function maximum(x, y, z) {
      return Math.max(x, Math.max(y, z));
    }
  </script>
</head><body></body>
</html>
```


Example 2

- Multiple function calls
- Intro to event-driven programming processing
- Displaying random images of dice

```
<html>
<head>
  <title>Random Dice Images</title>
  <style type = "text/css">
    li { display: inline; margin-right: 10px; }
    ul { margin: 0; }
  </style>
  <script>
    var dice1Image;
    var dice2Image;
    var dice3Image;
    var dice4Image;
    // register button listener and get the img elements
    function start() {
      var button = document.getElementById("rollButton");
      button.addEventListener("click", rollDice, false);
      dice1Image = document.getElementById("dice1");
      dice2Image = document.getElementById("dice2");
      dice3Image = document.getElementById("dice3");
      dice4Image = document.getElementById("dice4");
    } // end function rollDice
    // roll the dice
    function rollDice() {
      setImage(dice1Image);
      setImage(dice2Image);
      setImage(dice3Image);
      setImage(dice4Image);
    } // end function rollDice
    // set image source for a dice
    function setImage(diceImg) {
      var diceValue = Math.floor(1 + Math.random() * 6);
      diceImg.setAttribute("src", "dice" + diceValue + ".png");
      diceImg.setAttribute("alt", "dice image with " + diceValue + " spot(s)");
    } // end function setImage
    window.addEventListener("load", start, false);
  </script>
</head>
<body>
  <form action = "#">
    <input id = "rollButton" type = "button" value = "Roll Dice">
  </form>
  <ol>
    <li><img id = "dice1" src = "diceBlank.png" alt = "dice 1 image"></li>
    <li><img id = "dice2" src = "diceBlank.png" alt = "dice 2 image"></li>
    <li><img id = "dice3" src = "diceBlank.png" alt = "dice 3 image"></li>
    <li><img id = "dice4" src = "diceBlank.png" alt = "dice 4 image"></li>
  </ol>
</body>
</html>
```

Note: The CSS style for **li** places a margin of 10 pixels to the right of each li and indicates that they should display inline rather than vertically on the page



Notes

- Window object's load event
 - Event handler registered through method `addEventListener()`
 - The method takes three arguments
 - The name of the event for which we're registering a handler
 - The function that will be called to handle the event (function `start()` in this example)
 - The last argument is typically `false`
 - Function `start` registers the Roll Dice button's click event handler, which instructs the browser to listen for click events
- Note: Document object's `getElementById` method, finds the HTML5 element with the corresponding `id` attribute and returns a JavaScript object representing the element

Example 3

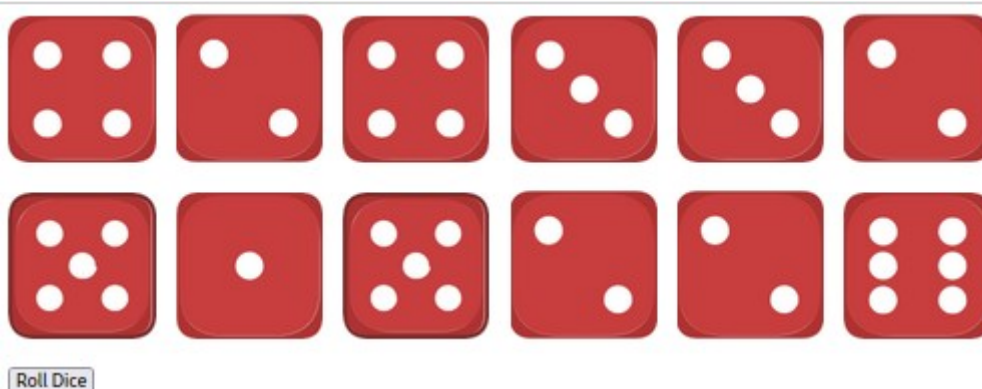
- Rolling Dice Repeatedly and Displaying Statistics

```
<html>
<head>
  <title>dice Rolling Frequencies</title>
  <style type = "text/css">
    img { margin-right: 10px; }
    table { width: 200px;
            border-collapse: collapse;
            background-color: lightblue; }
    table, td, th { border: 1px solid black;
                    padding: 4px;
                    margin-top: 20px; }
    th { text-align: left;
         color: white;
         background-color: darkblue; }
  </style>
<script>
  var frequency1 = 0;  var frequency2 = 0;  var frequency3 = 0;
  var frequency4 = 0;  var frequency5 = 0;  var frequency6 = 0;
  var totalDice = 0;
  // register button event handler
  function start() {
    var button = document.getElementById("rollButton");
    button.addEventListener("click", rollDice, false);
  }
  function rollDice() {
    var face; // face rolled
    // loop to roll dice 12 times
    for (var i = 1; i <= 12; ++i) {
      face = Math.floor(1 + Math.random() * 6);
      tallyRolls(face); // increment a frequency counter
      setImage(i, face); // display appropriate dice image
      ++totalDice; // increment total
    } // end dice rolling loop
    updateFrequencyTable();
  }
  // increment appropriate frequency counter
  function tallyRolls(face) {
    switch (face) {
      case 1:
        ++frequency1;  break;
      case 2:
        ++frequency2;  break;
      case 3:
        ++frequency3;  break;
      case 4:
        ++frequency4;  break;
      case 5:
        ++frequency5;  break;
      case 6:
        ++frequency6;  break;
    }
  }
}
```

```

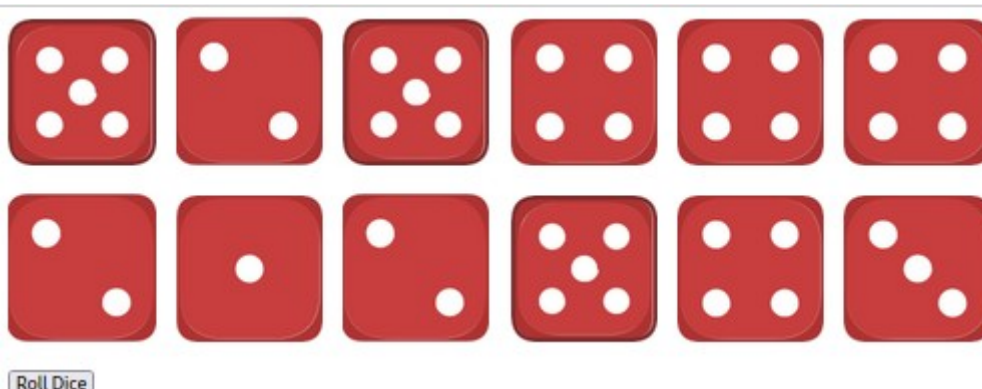
// set image source for a dice
function setImage(diceNumber, face) {
    var diceImg = document.getElementById("dice" + diceNumber);
    diceImg.setAttribute("src", "dice" + face + ".png");
    diceImg.setAttribute("alt", "dice with " + face + " spot(s)");
}
// update frequency table in the page
function updateFrequencyTable() {
    var tableDiv = document.getElementById("frequencyTableDiv");
    tableDiv.innerHTML = "<table>" +
        "<caption>dice Rolling Frequencies</caption>" +
        "<thead><th>Face</th><th>Frequency</th>" +
        "<th>Percent</th></thead>" +
        "<tbody><tr><td>1</td><td>" + frequency1 + "</td><td>" +
        formatPercent(frequency1 / totalDice) + "</td></tr>" +
        "<tr><td>2</td><td>" + frequency2 + "</td><td>" +
        formatPercent(frequency2 / totalDice) + "</td></tr>" +
        "<tr><td>3</td><td>" + frequency3 + "</td><td>" +
        formatPercent(frequency3 / totalDice) + "</td></tr>" +
        "<tr><td>4</td><td>" + frequency4 + "</td><td>" +
        formatPercent(frequency4 / totalDice) + "</td></tr>" +
        "<tr><td>5</td><td>" + frequency5 + "</td><td>" +
        formatPercent(frequency5 / totalDice) + "</td></tr>" +
        "<tr><td>6</td><td>" + frequency6 + "</td><td>" +
        formatPercent(frequency6 / totalDice) + "</td></tr>" +
        "</tbody></table>";
}
function formatPercent(value) {
    value *= 100;
    return value.toFixed(2);
}
window.addEventListener("load", start, false);
</script>
</head>
<body>
    <p><img id = "dice1" src = "diceBlank.png" alt = "dice 1 image">
    <img id = "dice2" src = "diceBlank.png" alt = "dice 2 image">
    <img id = "dice3" src = "diceBlank.png" alt = "dice 3 image">
    <img id = "dice4" src = "diceBlank.png" alt = "dice 4 image">
    <img id = "dice5" src = "diceBlank.png" alt = "dice 5 image">
    <img id = "dice6" src = "diceBlank.png" alt = "dice 6 image"></p>
    <p><img id = "dice7" src = "diceBlank.png" alt = "dice 7 image">
    <img id = "dice8" src = "diceBlank.png" alt = "dice 8 image">
    <img id = "dice9" src = "diceBlank.png" alt = "dice 9 image">
    <img id = "dice10" src = "diceBlank.png" alt = "dice 10 image">
    <img id = "dice11" src = "diceBlank.png" alt = "dice 11 image">
    <img id = "dice12" src = "diceBlank.png" alt = "dice 12 image"></p>
    <form action = "#">
    <input id = "rollButton" type = "button" value = "Roll Dice">
    </form>
    <div id = "frequencyTableDiv"></div>
</body>
</html>

```



dice Rolling Frequencies

Face	Frequency	Percent
1	1	8.33
2	4	33.33
3	2	16.67
4	2	16.67
5	2	16.67
6	1	8.33



dice Rolling Frequencies

Face	Frequency	Percent
1	2	8.33
2	7	29.17
3	3	12.50
4	6	25.00
5	5	20.83
6	1	4.17

5.4 Scope Rules

Scope of an identifier for a variable or function

- The portion of the program in which the identifier can be referenced
- Global scope
 - Functions or variables declared in the head element
 - Accessible in any part of a script
- Local scope (Function Scope)
 - Function parameters
 - Identifiers declared inside a function
 - Local variables of a function and function parameters have function scope.
 - Local variable in a function that has the same name as a global variable hides the global variable from the body of the function

```
<html>
<head>
  <title>Scoping Example</title>
  <style type = "text/css">
    p      { margin: 0px; }
    p.space { margin-top: 10px; }
  </style>
  <script>
    var output; // stores the string to display
    var x = 1; // global variable
    function start() {
      var x = 5; // variable local to function start
      output = "<p>local x in start is " + x + "</p>";
      functionA(); // functionA has local x
      functionB(); // functionB uses global variable x
      functionA(); // functionA reinitializes local x
      functionB(); // global variable x retains its value
      output += "<p class='space'>local x in start is " + x + "</p>";
      document.getElementById("results").innerHTML = output;
    } // end function start
    function functionA() {
      var x = 25; // initialized each time functionA is called
      output += "<p class='space'>local x in functionA is " + x + " after entering
functionA</p>";
      ++x;
      output += "<p>local x in functionA is " + x + " before exiting functionA</p>";
    } // end functionA
    function functionB() {
      output += "<p class='space'>global variable x is " + x + " on entering
functionB";
      x *= 10;
      output += "<p>global variable x is " + x + " on exiting functionB</p>";
    } // end functionB
    window.addEventListener("load", start, false);
  </script>
</head>
<body>
<div id = "results"></div>
</body>
</html>
```

local x in start is 5

local x in functionA is 25 after entering functionA

local x in functionA is 26 before exiting functionA

global variable x is 1 on entering functionB

global variable x is 10 on exiting functionB

local x in functionA is 25 after entering functionA

local x in functionA is 26 before exiting functionA

global variable x is 10 on entering functionB

global variable x is 100 on exiting functionB

local x in start is 5

5.5 JavaScript Arrays

5.5.1 Introduction

- Arrays
 - Data structures consisting of related data items
 - Sometimes called collections of data items
- JavaScript arrays
 - Dynamic entities that can change size after they are created
 - Like in other languages
 - An element may be referred to by giving the name of the array followed by index of the element in square brackets ([])
 - Zero-based indexes
 - Every array in JavaScript knows its own length, which it stores in its length attribute and can be found with the expression `arrayname.length`

Position number of the element within the array c

Name of the array is c

Name of an individual array element

Value of array element c[4]

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

5.5.2 Declaring and Allocating Arrays

- JavaScript arrays are Array objects
- Created using the **new** operator

5.5.3 Examples Using Arrays

Example 1

ArrResults.html

```
<html>
  <head>
    <title>Initializing an Array</title>
    <link rel = "stylesheet" type = "text/css" href = "tableStyle.css">
    <script src = "InitArray.js"></script>
  </head>
  <body>
    <div id = "output1"></div>
    <div id = "output2"></div>
  </body>
</html>
```

InitArray.js

```
function start() {
  var n1 = new Array(5); // allocate five-element array
  var n2 = new Array(); // allocate empty array
  // assign values to each element of array n1
  var length = n1.length; // get array's length once before the loop
  for (var i = 0; i < length; ++i) {
    n1[i] = i;
  } // end for
  // create and initialize five elements in array n2
  for (i = 0; i < 5; ++i) {
    n2[i] = 2*i;
  } // end for
  outputArray("Array n1:", n1, document.getElementById("output1"));
  outputArray("Array n2:", n2, document.getElementById("output2"));
}

function outputArray(heading, theArray, output) {
  var content = "<h2>" + heading + "</h2><table>" +
"<thead><th>Index</th><th>Value</th></thead><tbody>";
  // output the index and value of each array element
  var length = theArray.length; // get array's length once before loop
  for (var i = 0; i < length; ++i) {
    content += "<tr><td>" + i + "</td><td>" + theArray[i] + "</td></tr>";
  } // end for
  content += "</tbody></table>";
  output.innerHTML = content; // place the table in the output element
}
window.addEventListener("load", start, false);
```

- Note: Operator new dynamically allocates for the array (like with array n2 above)

tableStyle.css

```
table { width: 300px;
        border-collapse: collapse;
        background-color: lightblue; }
table, td, th { border: 1px solid black;
                padding: 4px; }
th          { text-align: left;
              color: white;
              background-color: darkblue; }
tr.oddrow   { background-color: white; }
```

Array n1:

Index	Value
0	0
1	1
2	2
3	3
4	4

Array n2:

Index	Value
0	0
1	2
2	4
3	6
4	8

Example 2: Using an Initializer List

If an array's element values are known in advance, the elements can be allocated and initialized in the declaration of the array

The array's size is determined by the number of values in the initializer list

Syntax

1. `var n = [ 10, 20, 30, 40, 50 ];`
2. `var n = new Array(10, 20, 30, 40, 50);`

3. `var n = [ 10, 20, , 40, 50 ];` //space reserved in array for a value to be specified later

ArrResults2.html

```
<html>
  <head>
    <title>Initializing an Array</title>
    <link rel = "stylesheet" type = "text/css" href = "tableStyle.css">
    <script src = "InitArray2.js"></script>
  </head>
  <body>
    <div id = "output1"></div>
    <div id = "output2"></div>
    <div id = "output3"></div>
  </body>
</html>
```

InitArray2.js

```
function start() {
  // Initializer lists
  var colors = new Array("cyan", "magenta", "yellow", "black");
  var integers1 = [ 2, 4, 6, 8 ];
  var integers2 = [ 2, , , 8 ];
  outputArray("Array colors contains", colors,
document.getElementById("output1"));
  outputArray("Array integers1 contains",
integers1, document.getElementById("output2"));
  outputArray("Array integers2 contains",
integers2, document.getElementById("output3"));
}

function outputArray(heading, theArray, output) {
  var content = "<h2>" + heading + "</h2><table>" +
"<thead><th>Index</th><th>Value</th></thead><tbody>";
  var length = theArray.length; // get array's length once before loop
  for (var i = 0; i < length; ++i) {
    content += "<tr><td>" + i + "</td><td>" + theArray[i] + "</td></tr>";
  }
  content += "</tbody></table>";
  output.innerHTML = content; // place the table in the output element
}
window.addEventListener("load", start, false);
```

Array colors contains

Index	Value
0	cyan
1	magenta
2	yellow
3	black

Array integers1 contains

Index	Value
0	2
1	4
2	6
3	8

Array integers2 contains

Index	Value
0	2
1	undefined
2	undefined
3	8

5.5.4 References and Reference Parameters

- Two ways to pass arguments to functions (or methods)
 - pass-by-value
 - pass-by-reference
- Pass-by-value
 - a copy of the argument's value is made and is passed to the called function
 - In JavaScript, numbers, boolean values and strings are passed to functions by value
- Pass-by-reference

- The caller gives the called function direct access to the caller's data and allows it to modify the data if it so chooses
- Can improve performance because it can eliminate the overhead of copying large amounts of data, but it can weaken security because the called function can access the caller's data
- All objects are passed to functions by reference
- Arrays are objects in JavaScript, so Arrays are passed to a function by reference
 - A called function can access the elements of the array elements
 - The name of the array is passed, and is a reference to an object that contains the array elements and the length variable

5.5.5 Passing Arrays to Functions

- Pass an array as an argument to a function
- Specify the name of the array
- To pass an array element to a function, use the subscripted name of the element as an argument in the function call

ArrResults3.html

```
<html>
  <head>
    <title>Initializing an Array</title>
    <link rel = "stylesheet" type = "text/css" href = "tableStyle.css">
    <script src = "PassArray.js"></script>
  </head>
  <body>
    <h2>Effects of passing entire array by reference</h2>
    <p id = "originalArray"></p>
    <p id = "modifiedArray"></p>
    <h2>Effects of passing array element by value</h2>
    <p id = "originalElement"></p>
    <p id = "inModifyElement"></p>
    <p id = "modifiedElement"></p>
  </body>
</html>
```

PassArray.js

```
function start() {
    var a = [1, 2, 3, 4, 5];
    outputArray("Original array: ", a, document.getElementById("originalArray"));
    modifyArray(a); // array a passed by reference
    outputArray("modified array: ", a, document.getElementById("modifiedArray"));
    document.getElementById("originalElement").innerHTML = "a[3] before modifyElement: "
    + a[3];
    modifyElement(a[3]); // array element a[3] passed by value
    document.getElementById("modifiedElement").innerHTML = "a[3] after modifyElement: "
    + a[3];
}

function outputArray(heading, theArray, output) {
    output.innerHTML = heading + theArray.join(" ");
}

function modifyArray(theArray) {
    for (var j in theArray) {
        theArray[j] *= 2;
    }
}

function modifyElement(e) {
    e *= 2; // scales element e only for the duration of the function
    document.getElementById("inModifyElement").innerHTML = "Value in modifyElement: "
    + e;
}

window.addEventListener("load", start, false);
```

Effects of passing entire array by reference

Original array: 1 2 3 4 5

Modified array: 2 4 6 8 10

Effects of passing array element by value

a[3] before modifyElement: 8

Value in modifyElement: 16

a[3] after modifyElement: 8

5.5.6 Sorting Arrays

- Array object in JavaScript has a built-in method **sort**
 - With no arguments, the method uses string comparisons to determine the sorting order of the Array elements
 - Method sort takes as its optional argument the name of a function (called the comparator function) that compares its two arguments and returns a negative value, zero, or a positive value, if the first argument is less than, equal to, or greater than the second, respectively
 - Functions in JavaScript are considered to be data
 - They can be assigned to variables, stored in Arrays and passed to functions just like other data

ArrResults4.html

```
<html>
  <head>
    <title>Array Method sort</title>
    <link rel = "stylesheet" type = "text/css" href = "tableStyle.css">
    <script src = "Sort.js"></script>
  </head>
  <body>
    <h1>Sorting an Array</h1>
    <p id = "originalArray"></p>
    <p id = "sortedArray"></p>
  </body>
</html>
```

Sort.js

```
function start() {
  var a = [10, 1, 9, 2, 8, 3, 7, 4, 6, 5];
  outputArray("Data items in original order: ", a,
    document.getElementById("originalArray"));
  a.sort(compareIntegers); // sort the array
  outputArray("Data items in ascending order: ", a,
    document.getElementById("sortedArray"));
}
function outputArray(heading, theArray, output) {
  output.innerHTML = heading + theArray.join(" ");
}
function compareIntegers(value1, value2) {
  return parseInt(value1) - parseInt(value2);
}
window.addEventListener("load", start, false);
```

- join method of an array
 - Returns a string that contains all of the elements of an array, separated by the string supplied in the function's argument
 - If an argument is not specified, the empty string is used as the separator

Sorting an Array

Data items in original order: 10 1 9 2 8 3 7 4 6 5

Data items in ascending order: 1 2 3 4 5 6 7 8 9 10

Without comparator, the output will be:

Sorting an Array

Data items in original order: 10 1 9 2 8 3 7 4 6 5

Data items in ascending order: 1 10 2 3 4 5 6 7 8 9

5.5.7 Searching Arrays with Array Method indexOf

ArrResults5.html

```
<html>
  <head>
    <title>Search an Array</title>
    <script src = "search.js"></script>
  </head>
  <body>
    <form action = "#">
      <p><label>Enter integer search key:
        <input id = "inputVal" type = "number"></label>
        <input id = "searchButton" type = "button" value = "Search">
      </p>
      <p id = "result"></p>
    </form>
  </body>
</html>
```


search.js

```
var a = new Array(100);
for (var i = 0; i < a.length; ++i) {
    a[i] = 2 * i;
}
function buttonPressed() {
    // get the input text field
    var inputVal = document.getElementById("inputVal");
    // get the result paragraph
    var result = document.getElementById("result");
    // get the search key from the input text field then perform the search
    var searchKey = parseInt(inputVal.value);
    var element = a.indexOf(searchKey);
    if (element != -1) {
        result.innerHTML = "Found value in element " + element;
    }
    else {
        result.innerHTML = "Value not found";
    }
}
// register searchButton's click event handler
function start() {
    var searchButton = document.getElementById("searchButton");
    searchButton.addEventListener("click", buttonPressed, false);
}
window.addEventListener("load", start, false);
```

Enter integer search key:

Found value in element 1

5.5.8 Multidimensional Arrays

See text for sample code.

5.6 JavaScript Objects

ArrResults5.html

```
<html>
  <head>
    <title>Search an Array</title>
    <script src = "search.js"></script>
  </head>
  <body>
    <form action = "#">
      <p><label>Enter integer search key:
        <input id = "inputVal" type = "number"></label>
        <input id = "searchButton" type = "button" value = "Search">
      </p>
      <p id = "result"></p>
    </form>
  </body>
</html>
```

5.6.1 Introduction

- Objects have attributes and exhibit behaviors
- Introducing several of JavaScript's built-in objects
- Objects have the property of information hiding
 - Objects may know how to communicate with one another across well-defined interfaces, but normally they are not allowed to know how other objects are implemented
- Web browsers
 - Contain a set of objects that encapsulate an XHTML document's elements
 - The objects expose to a JavaScript programmer the attributes and behaviours that enable a JavaScript program to interact with (or script) those elements (objects)

5.6.2 Math Object

- Math object methods allow you to perform many common mathematical calculations
- An object's methods are called by
 - Writing the name of the object followed by a dot operator (.) and the name of the method
 - In parentheses following the method name is the argument (or a comma-separated list of arguments) to the method
 - Example
 - `var result = Math.sqrt(900);`

Math object methods

Method	Description	Examples
<code>abs(x)</code>	Absolute value of x.	<code>abs(7.2)</code> is 7.2 <code>abs(0)</code> is 0 <code>abs(-5.6)</code> is 5.6
<code>ceil(x)</code>	Rounds x to the smallest integer not less than x.	<code>ceil(9.2)</code> is 10 <code>ceil(-9.8)</code> is -9.0
<code>cos(x)</code>	Trigonometric cosine of x (x in radians).	<code>cos(0)</code> is 1
<code>exp(x)</code>	Exponential method e^x .	<code>exp(1)</code> is 2.71828 <code>exp(2)</code> is 7.38906
<code>floor(x)</code>	Rounds x to the largest integer not greater than x.	<code>floor(9.2)</code> is 9 <code>floor(-9.8)</code> is -10.0
<code>log(x)</code>	Natural logarithm of x (base e).	<code>log(2.718282)</code> is 1 <code>log(7.389056)</code> is 2
<code>max(x, y)</code>	Larger value of x and y.	<code>max(2.3, 12.7)</code> is 12.7 <code>max(-2.3, -12.7)</code> is -2.3
<code>min(x, y)</code>	Smaller value of x and y.	<code>min(2.3, 12.7)</code> is 2.3 <code>min(-2.3, -12.7)</code> is -12.7
<code>pow(x, y)</code>	x raised to power y (x^y).	<code>pow(2, 7)</code> is 128 <code>pow(9, .5)</code> is 3.0
<code>round(x)</code>	Rounds x to the closest integer.	<code>round(9.75)</code> is 10 <code>round(9.25)</code> is 9
<code>sin(x)</code>	Trigonometric sine of x (x in radians).	<code>sin(0)</code> is 0
<code>sqrt(x)</code>	Square root of x.	<code>sqrt(900)</code> is 30 <code>sqrt(9)</code> is 3
<code>tan(x)</code>	Trigonometric tangent of x (x in radians).	<code>tan(0)</code> is 0

Math object properties

Constant	Description	Value
<code>Math.E</code>	Base of a natural logarithm (e).	Approximately 2.718
<code>Math.LN2</code>	Natural logarithm of 2.	Approximately 0.693
<code>Math.LN10</code>	Natural logarithm of 10.	Approximately 2.302
<code>Math.LOG2E</code>	Base 2 logarithm of e .	Approximately 1.442
<code>Math.LOG10E</code>	Base 10 logarithm of e .	Approximately 0.434
<code>Math.PI</code>	π —the ratio of a circle's circumference to its diameter.	Approximately 3.141592653589793
<code>Math.SQRT1_2</code>	Square root of 0.5.	Approximately 0.707
<code>Math.SQRT2</code>	Square root of 2.0.	Approximately 1.414

5.6.3 String Object

- A series of characters treated as a single unit
- A string may include letters, digits and various special characters, such as +, -, *, /, and \$
- JavaScript supports Unicode, which represents a large portion of the world's languages
- String literals or string constants (often called anonymous String objects) are written as a sequence of characters in double quotation marks or single quotation marks

JavaScript String methods

JavaScript String Methods



- | | | |
|---|-------------------------------|--------------------------------|
| 1. <code>charAt()</code> | 9. <code>matchAll()</code> | 17. <code>substr()</code> |
| 2. <code>charCodeAt()</code> | 10. <code>repeat()</code> | 18. <code>substring()</code> |
| 3. <code>concat(str1, str2, ...)</code> | 11. <code>replace()</code> | 19. <code>toLowerCase()</code> |
| 4. <code>includes()</code> | 12. <code>replaceAll()</code> | 20. <code>toUpperCase()</code> |
| 5. <code>endsWith()</code> | 13. <code>search()</code> | 21. <code>toString()</code> |
| 6. <code>indexOf()</code> | 14. <code>slice()</code> | 22. <code>trim()</code> |
| 7. <code>lastIndexOf()</code> | 15. <code>split()</code> | 23. <code>valueOf()</code> |
| 8. <code>match()</code> | 16. <code>startsWith()</code> | |

TutorialsTonight.com

Character Processing Methods

- String method `charAt`
 - Returns the character at a specific index in a string
 - Indices for the characters in a string start at 0 (the first character) and go up to (but do not include) the string's length
 - If the index is outside the bounds of the string, the method returns an empty string
- String method `charCodeAt`
 - Returns the Unicode value of the character at a specific index in a string
 - If the index is outside the bounds of the string, the method returns NaN
- String method `fromCharCode`
 - Creates a string from a list of Unicode values
- String method `toLowerCase`
 - Returns the lowercase version of a string
- String method `toUpperCase`
 - Returns the uppercase version of a string

charProcessing.html

```
<html>
  <head>
    <title>Character Processing</title>
    <script src = "charProcessing.js"></script>
  </head>
  <body>
    <div id = "results"></div>
  </body>
</html>
```

charProcessing.html

```
function start() {
  var s = "ZEBRA";
  var s2 = "AbCdEfG";
  var result = "";
  result = "<p>Character at index 0 in '" + s + "' is " +
    s.charAt(0) + "</p>";
  result += "<p>Character code at index 0 in '" + s + "' is " +
    s.charCodeAt(0) + "</p>";
  result += "<p>'" + String.fromCharCode(87, 79, 82, 68) +
    "' contains character codes 87, 79, 82 and 68</p>";
  result += "<p>'" + s2 + "' in lowercase is '" +
    s2.toLowerCase() + "'</p>";
  result += "<p>'" + s2 + "' in uppercase is '" +
    s2.toUpperCase() + "'</p>";
  document.getElementById("results").innerHTML = result;
}
window.addEventListener("load", start, false);
```

String search

stringSearch.html

```
<html>
  <head>
    <title>Searching Strings</title>
    <script src = "stringSearch.js"></script>
  </head>
  <body>
    <form id = "searchForm" action = "#">
      <h1>The string to search is:
abcdefghijklmnopqrstuvwxyzabcdefghijklm</h1>
      <p>Enter the substring to search for
      <input id = "inputfield" type = "search">
      <input id = "searchButton" type = "button" value = "Search"></p>
      <div id = "results"></div>
    </form>
  </body>
</html>
```

stringSearch.js

```
var letters = "abcdefghijklmnopqrstuvwxyzabcdefghijklm";
function buttonPressed() {
  var inputfield = document.getElementById("inputfield");
  document.getElementById("results").innerHTML =
    "<p>first occurrence is located at index " +
    letters.indexOf(inputfield.value) + "</p>" +
    "<p>Last occurrence is located at index " +
    letters.lastIndexOf(inputfield.value) + "</p>" +
    "<p>first occurrence from index 12 is located at index " +
    letters.indexOf(inputfield.value, 12) + "</p>" +
    "<p>Last occurrence from index 12 is located at index " +
    letters.lastIndexOf(inputfield.value, 12) + "</p>";
}

function start() {
  var searchButton = document.getElementById("searchButton");
  searchButton.addEventListener("click", buttonPressed, false);
}
window.addEventListener("load", start, false);
```

The string to search is: abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz

Enter the substring to search for

First occurrence is located at index 3

Last occurrence is located at index 29

First occurrence from index 12 is located at index 29

Last occurrence from index 12 is located at index 3

The string to search is: abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz

Enter the substring to search for

First occurrence is located at index 23

Last occurrence is located at index 23

First occurrence from index 12 is located at index 23

Last occurrence from index 12 is located at index -1

Splitting Strings and Obtaining Substrings

stringSplit.html

```
<html>
  <head>
    <title>split and substring</title>
    <script src = "SplitAndSubString.js"></script>
  </head>
  <body>
    <form action = "#">
      <p>Enter a sentence to split into words:</p>
      <p><input id = "inputfield" type = "text">
      <input id = "splitButton" type = "button" value = "Split"></p>
      <div id = "results"></div>
    </form>
  </body>
</html>
```

split

stringSplit.js

```
function splitButtonPressed() {
  var inputString = document.getElementById("inputfield").value;
  var tokens = inputString.split(" ");
  var results = document.getElementById("results");
  results.innerHTML = "<p>The sentence split into words is: </p>" +
    "<p class = 'indent'>" +
    tokens.join("</p><p class = 'indent'>") + "</p>" +
    "<p>The first 10 characters of the input string are: </p>" +
    "<p class = 'indent'>" + inputString.substring(0, 10) + "'</p>";
}
function start() {
  var splitButton = document.getElementById("splitButton");
  splitButton.addEventListener("click", splitButtonPressed, false);
}
window.addEventListener("load", start, false);
```

form action = "#"

Enter a sentence to split into words:

This sentence is tokenized below

Split

The sentence split into words is:

This

sentence

is

tokenized

below

The first 10 characters of the input string are:

'This sente'

5.6.4 Date Object

- Date object provides methods for date and time manipulations
 - Based either on the computer's local time zone or on World Time Standard's Coordinated Universal Time (abbreviated UTC)
- Most methods have a local time zone and a UTC version
- Empty parentheses after an object name indicate a call to the object's constructor with no arguments
 - A constructor is an initializer method for an object
 - Called automatically when an object is allocated with new
 - The Date constructor with no arguments initializes the Date object with the local computer's current date and time
 - A new Date object can be initialized by passing the number of milliseconds since midnight, January 1, 1970, to the Date constructor
 - Can also create a new Date object by supplying arguments to the Date constructor for year, month, date, hours, minutes, seconds and milliseconds
 - Hours, minutes, seconds and milliseconds arguments are all optional
 - If any one of these arguments is not specified, a zero is supplied in its place
 - If an argument is specified, all arguments to its left must be specified

date.html

```
<html>
  <head>
    <title>Date and Time Methods</title>
    <script src = "DateTime.js"></script>
  </head>
  <body>
    <h1>String representations and valueOf</h1>
    <section id = "strings"></section>
    <h1>Get methods for local time zone</h1>
    <section id = "getMethods"></section>
    <h1>Specifying arguments for a new Date</h1>
    <section id = "newArguments"></section>
    <h1>Set methods for local time zone</h1>
    <section id = "setMethods"></section>
  </body>
</html>
```

DateTime.js

```
function start() {
    var current = new Date();
    // string-formatting methods and valueOf
    document.getElementById("strings").innerHTML =
        "<p>toString: " + current.toString() + "</p>" +
        "<p>toLocaleString: " + current.toLocaleString() + "</p>" +
        "<p>toUTCString: " + current.toUTCString() + "</p>" +
        "<p>valueOf: " + current.valueOf() + "</p>";
    // get methods
    document.getElementById("getMethods").innerHTML =
        "<p>getDate: " + current.getDate() + "</p>" +
        "<p>getDay: " + current.getDay() + "</p>" +
        "<p>getMonth: " + current.getMonth() + "</p>" +
        "<p>getFullYear: " + current.getFullYear() + "</p>" +
        "<p>getTime: " + current.getTime() + "</p>" +
        "<p>getHours: " + current.getHours() + "</p>" +
        "<p>getMinutes: " + current.getMinutes() + "</p>" +
        "<p>getSeconds: " + current.getSeconds() + "</p>" +
        "<p>getMilliseconds: " + current.getMilliseconds() + "</p>" +
        "<p>getTimezoneOffset: " + current.getTimezoneOffset() + "</p>";
    // creating a Date
    var anotherDate = new Date(2011, 2, 18, 1, 5, 0, 0);
    document.getElementById("newArguments").innerHTML =
        "<p>Date: " + anotherDate + "</p>";
    // set methods
    anotherDate.setDate(31);          anotherDate.setMonth(11);
    anotherDate.setFullYear(2011);    anotherDate.setHours(23);
    anotherDate.setMinutes(59);        anotherDate.setSeconds(59);
    document.getElementById("setMethods").innerHTML =
        "<p>modified date: " + anotherDate + "</p>";
}
window.addEventListener("load", start, false);
```

String representations and valueOf

toString: Wed Apr 19 2023 12:39:21 GMT+0100 (West Africa Standard Time)

toLocaleString: 4/19/2023, 12:39:21 PM

toUTCString: Wed, 19 Apr 2023 11:39:21 GMT

valueOf: 1681904361626

Get methods for local time zone

getDate: 19

getDay: 3

getMonth: 3

getFullYear: 2023

getTime: 1681904361626

getHours: 12

getMinutes: 39

getSeconds: 21

getMilliseconds: 626

getTimezoneOffset: -60

Specifying arguments for a new Date

Date: Fri Mar 18 2011 01:05:00 GMT+0100 (West Africa Standard Time)

Set methods for local time zone

Modified date: Sat Dec 31 2011 23:59:59 GMT+0100 (West Africa Standard Time)

5.6.5 Boolean and Number Objects

See text

5.6.6 document Object

- For manipulating the document that is currently visible in the browser window

Method or property	Description
<code>getElementById(id)</code>	Returns the DOM node representing the XHTML element whose <code>id</code> attribute matches <i>id</i> .
<code>write(string)</code>	Writes the string to the XHTML document as XHTML code.
<code>writeln(string)</code>	Writes the string to the XHTML document as XHTML code and adds a newline character at the end.
<code>cookie</code>	A string containing the values of all the cookies stored on the user's computer for the current document. See Section 11.9, Using Cookies.
<code>lastModified</code>	The date and time that this document was last modified.

5.6.7 window Object

- Provides methods for manipulating browser windows
- window object open method
 - Creates a window
 - Three parameters
 - The URL of the page to open in the new window
 - The name of the window
 - A string of comma-separated, all-lowercase feature names, each followed by an = sign and either "yes" or "no" to determine whether that feature should be displayed in the new window
 - If these parameters are omitted, the browser defaults to a new window containing an empty page, no title and all features visible
- window object closed property
 - Contains a boolean value that is true if the window is closed and false if the window is open
- close method
 - Closes the current window and deletes its object from memory
- location property
 - contains a string representation of the URL displayed in the current window